
Distributed Systems

International Institute of Information Technology

Hyderabad, India

Pictures not cited explicitly are taken from the course book or wikipedia.
Slides are edited from various sources included Pallabh Dasguptas Slides

Course Policies

Textbook(s)

- Distributed Computing: Principles, Algorithms, and Systems: Kshemkalyani and Singhal.
- Distributed Algorithms: Gerald Tel, Nancy Lynch

Grading policy – Almost Final!

- Slip Tests: 10
- Project: 20
- Assignments : 25
- Quiz-2(no quiz 1) + Midsem + Endsem: $(10+12)+23=45$

Motivation

A group of computers working together as to appear as a single computer to the end-user. – Tannenbaum

Eg: Airline reservation, Online Banking

Required Features:

Lot of computers

Perform Concurrently

Fail independently

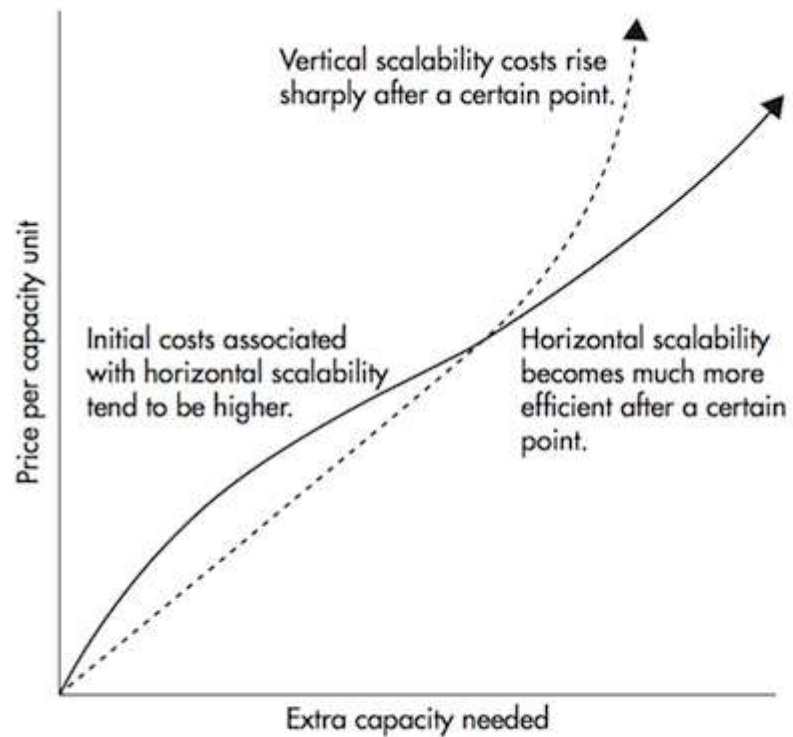
Dont share a global clock

Why Distributed Systems?

Best hardware may be insufficient to scale vertically (upgrade hardware of a system) hence the need to scale horizontally. Other benefits: ***Fault tolerance*** and ***low latency***.

Motivation

- Horizontal Scaling to fit data, shift data to manage load - scalability
- Inherently distributed. Ex: Banking system across cities
- Data Sharing. Ex Libraries, certain files, Complete datasets
- Enhanced Reliability
- Increased Performance/cost ratio (parallelization)
- Fault Tolerance

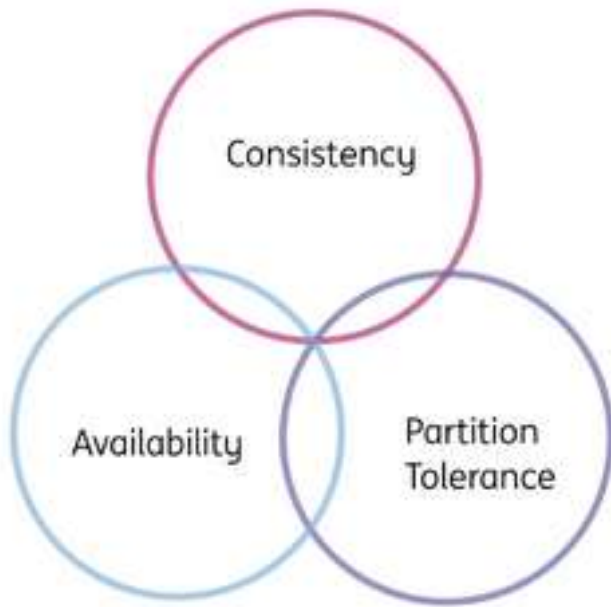


Horizontal scaling becomes **much cheaper** after a certain threshold

Challenges with a Distributed System

- Unreliable communication – messages may drop, could be intercepted, could be delayed, could be out of order.
- Lack of global knowledge – a participating node has information of only his local memory and local computations. (no shared memory)
- Lack of synchronisation (different local clocks)-how to implement sequence in which we want things to happen; how to order steps across participating nodes where each node has a different clock
- Concurrency control – Data distributed across nodes. Moreover each has its own clock. How do you implement Mutual exclusion now or critical sections now?
- Failure and recovery : Nodes may fail, communication channels may fail – how do you still make sure the system functions. When a node recovers how do you make the updates on it.
- Deadlocks, Termination Detection, File systems are other concerns

CAP Theorem- Eric Brewer (briefly)



The theorem states that networked shared-data systems can only guarantee/strongly support two of the following three properties:

Consistency — A guarantee that every node in a distributed cluster returns the same, most recent, successful write. Consistency refers to every client having the same view of the data.

Availability — Every non-failing node must be able to respond for all read and write requests in a reasonable amount of time.

Partition Tolerant — The system continues to function in spite of network partitions.

Network partitions and dropped messages are a fact of life and must be handled forcing designers to either choose perfect consistency or perfect availability.

Can we solve these -

The village without a mirror – consensus

War Preparation with messengers

- Consensus: All non-faulty processes must agree on a common value.
If initial value of all non-faulty processes is v , then the agreed upon value must be v

Consensus with traitors included - the general true then all true ppl will have same value but general too may be malicious (Byzantine Agreement Problem) – still all true ppl should decide same value not knowing who are are malicious –how ?

Time in Distributed Systems

Why is Time a Challenge?

IIIT Organises a coding competition. First prize to the first person who sends in the working code that runs on all test cases first and so on.

Raghav sent it at time t_1 and it reached me at time t_2
Sunita sent it at t_3 and it reached me at t_4 .

Now who is first ?

Obviously $t_2 < t_4$ does not imply $t_1 < t_3$

Why is Time a Challenge?

IIIT Organises a coding competition. First prize to the first person who sends in the working code that runs on all test cases first and so on.

Raghav sent it at time t_1 and it reached me at time t_2
Sunita sent it at t_3 and it reached me at t_4 .

Now who is first ?

Obviously $t_2 < t_4$ does not imply $t_1 < t_3$

Make sure Raghav and Sunita have their clocks set to the same time

Make sure both work with reference to a global time

Design something that inspite of their local clocks; I can still figure out who sent it first

Why is Time a Challenge?

Another example:

Consider the need to execute Mutual exclusion so that a banking transaction completes. A starts to transfer 1lakh to B and node B needs to know $A+B$.

Hence I need to apply the locks/semaphores.

How do I order these events such that B's query to know $A+B$ does not get executed midway A sending and B receiving.

This Lecture

- Understand the notion of time in distributed systems.
- How to maintain clocks in a distributed system.
 - Physical vs. Logical time
- Various algorithms/protocols for maintaining logical time.
- Snapshots and their uses
 - How to take a snapshot? Consistency of a snapshot.

Time in Distributed Systems

- Recall the quartz oscillation based clock ticks used in digital systems.
 - These clocks drift.
- With such a drift, it is not possible to arrange events according to a causal order.

Why is time important

- to determining the order in which events occur
- helps ensure liveness and fairness in mutual exclusion algorithms
- helps maintain consistency in replicated databases,
- helps design correct deadlock detection algorithms to avoid phantom and undetected deadlocks.
- Tracking of dependent events
- In debugging, it helps construct a consistent state
- for resuming reexecution; in failure recovery, it helps build a checkpoint;
- in replicated databases,
- it aids in the detection of file inconsistencies in case of a network partitioning

Motivation

- To ensure a correct and consistent view of a distributed database.
- Sensor networks and sensing properties of the physical world.
- Applications using time-outs to deduce certain actions/events.

Physical Time and Synchronization

- In centralized systems, there is only single clock. A process gets the time by simply issuing a system call to the kernel.
 - The time returned by the kernel is the “time”.
- In distributed systems, there is no global clock or common memory. Each processor has its own internal clock and its own notion of time.
- These clocks can easily drift seconds per day, accumulating significant errors over time.
- This clearly poses serious problems to applications that depend on a synchronized notion of time.

Cristian's (Time Server) Algorithm

- Uses a *time server* to synchronize clocks
 - Time server keeps the reference time (say UTC)
 - A client asks the time server for time, the server responds with its current time, and the client uses the received value T to set its clock
- But network round-trip time introduces errors...
 - Let **RTT = response-received-time – request-sent-time** (measurable at client),
 - If we know (a) $\min$ = minimum client-server one-way transmission time and (b) that the server timestamped the message at the last possible instant before sending it back
 - Then, the actual time could be between **$[T + \min, T + \text{RTT} - \min]$** where $\text{RTT} - \min$ is the max time it took for the response to return which happens when request took $\min$ time to go

Berkeley UNIX algorithm

- One master machine and set of slave nodes
- Periodically, this daemon polls and asks all the machines for their time
- The machines respond.
- The daemon computes an average time and then broadcasts this average time.

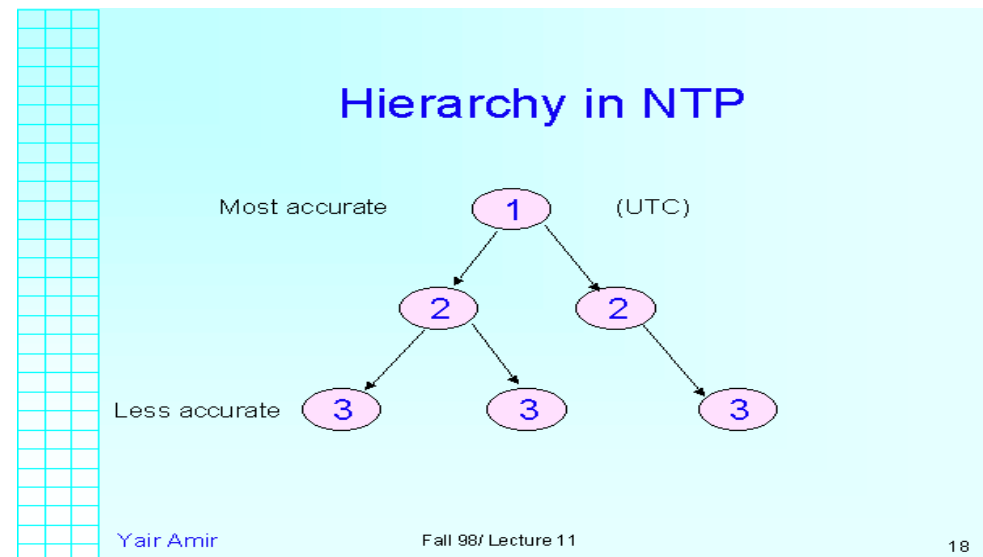
The algorithm assumes that each machine node in the network either doesn't have an accurate time source or doesn't possess an UTC server.

Decentralized Averaging Algorithm

- Each machine has a daemon without UTC
- Periodically, at fixed agreed-upon times, each machine broadcasts its local time.
- Each of them calculates the average time by averaging all the received local times.

Network Time Protocol (NTP)

- Most widely used physical clock synchronization protocol on the Internet
- 10-20 million NTP servers and clients in the Internet
- Claimed Accuracy (Varies)
 - milliseconds on WANs, submilliseconds on LANs
- Hierarchical tree of time servers.
 - The primary server at the root synchronizes with the UTC.
 - Secondary servers - backup to primary server.
 - Lowest
 - synchronization subnet with clients.



-
- However even with NTP, physical time cannot be perfectly synchronised. Moreover different nodes might have different NTP servers they are talking to as well.

But, do we really need to know the physical time or do we only need to know the sequence of events and that too among dependent events?

Modeling Distributed Systems

- Can think of a distributed system as a collection of processors, and
- A communication network.
- The processors
 - May fail
 - Share no global memory
- We also assume that the network
 - Has delays that are finite but unpredictable.
 - May not respect message ordering
 - Can lose messages, duplicate messages
 - May have links that become unavailable/may fail.

A Distributed Program

distributed program is

- A set of n processes, $P_1, P_2, \dots, P_n$, typically each running on a different processor.
 - A set of channels C_{ij} that connects Processor i to Processor j .
 - The state of a process P_i includes
 - The local memory of P_i
 - Also depends on messages sent,
- The state of a channel C_{ij} includes
- The messages in transit on this channel.

Types of events

- ❖ We will assume that processor sending a message does not wait for its delivery.
- ❖ Any distributed program has three types of events
 - ❖ Local actions
 - ❖ Message Send
 - ❖ Message Receive
- ❖ Can then view a distributed program as a sequential execution of the above events.

A Distributed Program in Execution

- Events can change the state of one or more process as follows.
 - A local action, aka an internal event, changes the state of the process where the event occurs
 - A send event, denoted $\text{send}(m)$ for message m , changes the state of the process sending the message and the state of the channel that is carrying the message m .
 - A receive event, denoted $\text{recv}(m)$ for message m , changes the state of the process that receives the message m and the state of the channel on which the message is received.

Logical Time

- Logical time is a notion we introduce to order events. It provides a mechanism to define the *causal order* in which events occur at different processes. The ordering is based on the following:
- Two events occurring at the same process happen in the order in which they are observed by the process.
- If a message is sent from one process to another, the sending of the message happened before the receiving of the message.
- If e occurred before e' and e' occurred before e'' then e occurred before e'' .

Causality between events

- Think of events happening at a process P_i .
- One can order these events sequentially,
- In other words, place a linear order on these events.
- Let $h_i = \{e^1_i, e^2_i, \dots\}$, be the events at process P_i .
- A linear order H_i is a binary relation $\rightarrow_i$ on the events h_i such that $e^k_i \rightarrow_i e^j_i$ if and only if the event e^k_i occurs before the event e^j_i at Process P_i .
- The dependencies captured by $\rightarrow_i$ are often called as causal dependencies among the events h_i at P_i .

Causality for msgs

- What about events generated due to messages?
- Consider a binary relation $\rightarrow_{\text{msg}}$ across messages exchanged.
- Clearly, for any message m , $\text{send}(m) \rightarrow_{\text{msg}} \text{recv}(m)$.
- These two relations $\rightarrow_i$ and $\rightarrow_{\text{msg}}$ allow us to view the execution of a distributed program

Causality Definition

- One can now view the execution of a distributed program as a collection of events.
- Consider the set of events $H = \bigcup_i h_i$, where h_i is the events that occurred at process P_i .
- Define a binary relation $\rightarrow$ expressing causality among pairs of events that possibly occur at different processes. $\rightarrow$ defined as:

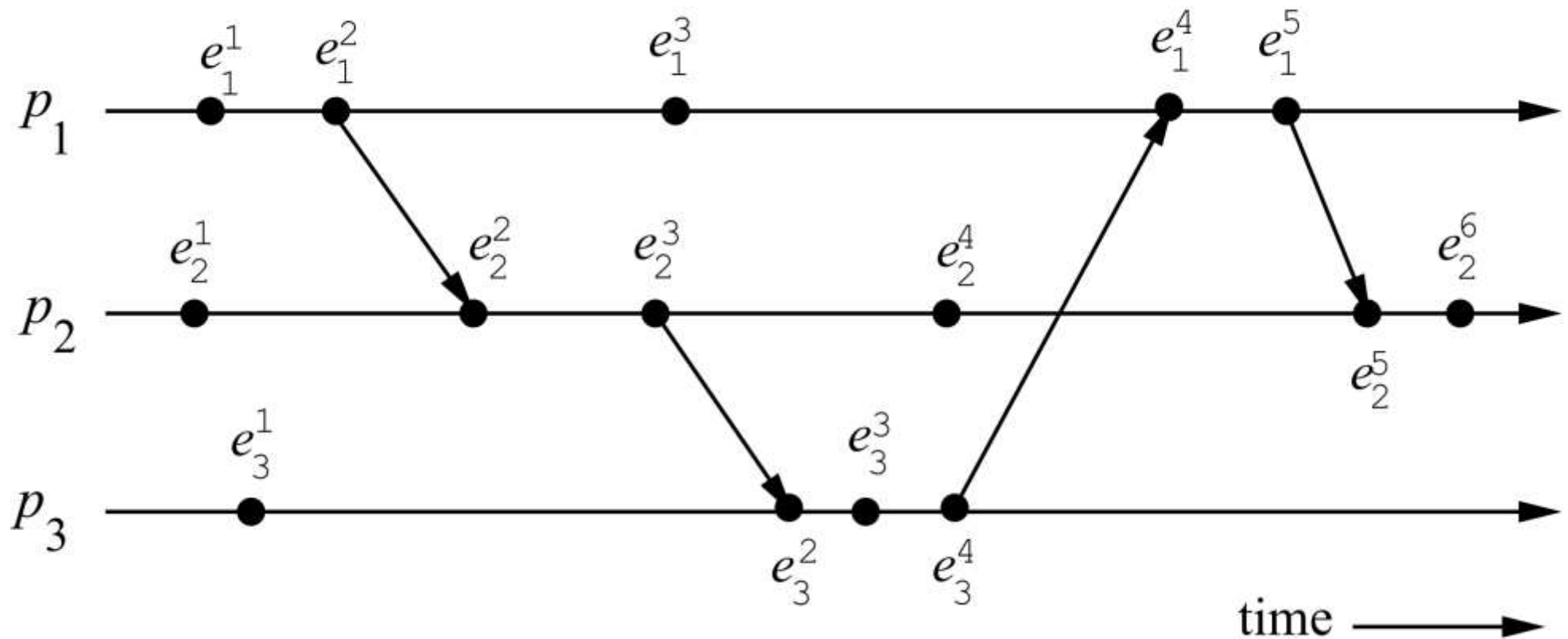
$$e^x_i \rightarrow e^y_j \text{ iff } \left\{ \begin{array}{l} i = j \text{ and } x < y, \quad \text{or} \\ e^x_i \rightarrow_{\text{msg}} e^y_j, \quad \text{or} \\ \text{There exists } e^z_k \text{ in } H \text{ s.t. } e^x_i \rightarrow e^z_k \text{ and } e^z_k \rightarrow e^y_j \end{array} \right.$$

Logical vs. Physical Concurrency

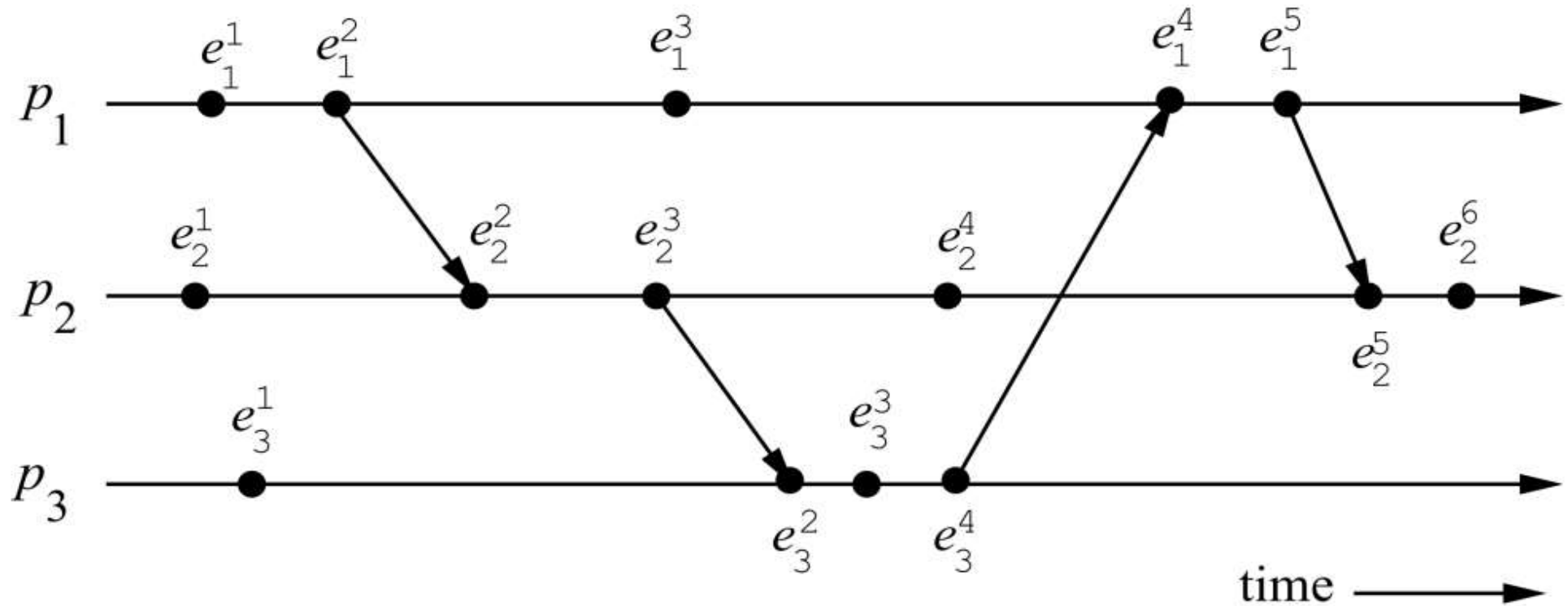
- Two events are **logically concurrent** if and only if the events do not causally affect each other. In other words, $e_i \parallel e_j \leftrightarrow \text{Not}(e_i \rightarrow e_j) \text{ and } \text{Not}(e_j \rightarrow e_i)$.
- Note that for logical concurrency of two events, the events may not occur at the same time.
- On the other hand, we define that two events are **physically concurrent** iff the events occur at the same physical time.

A Distributed Program in Execution

- Examples – Find concurrent events, Find some non-trivial (across P_i s) precedence among events.



A Distributed Program in Execution



- Concurrent: e_3^1 and e_1^3 ? e_2^4 and e_1^3 ?
- Causal: $e_3^3 \rightarrow e_1^5$, $e_1^2 \rightarrow e_2^3$, $e_3^4 \rightarrow e_1^5$?

Logical Time

- Armed with our notation of events and precedences amongst events, we now study logical time.
- We now see how logical time can be maintained in a distributed system.
- Three ways to implement logical time -
 - scalar time,
 - vector time, and
 - matrix time

Logical Time

- The logical clock C is a function that maps an event e in a distributed system to an element in the time domain T , denoted as $C(e)$ and called the timestamp of e , and is defined as follows:

$$C : H \rightarrow T$$

such that for two events e_i and e_j , $e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$.

Elements of T form a partially ordered or totally ordered set over a relation $<$.

Logical Time

- **Consistent:** When T and C satisfy the condition: for two events e_i and e_j , $e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$.
This monotonicity property is called the clock **consistency** condition.
- **Strongly Consistent:** When T and C satisfy the condition: for two events e_i and e_j , $e_i \rightarrow e_j \Leftrightarrow C(e_i) < C(e_j)$.
then the system of clocks is said to be strongly consistent.

Logical Time

- Each processor needs some data structure to represent logical time.
 - A local logical clock, denoted by lc_i , that helps process p_i measure its own progress.
 - A logical global clock, denoted by gc_i , that is a representation of process p_i 's local view of the logical global time.
 - Allows p_i to assign consistent time stamps to its local events.
 - Typically, lc_i is a part of gc_i .
- Each processor needs a protocol to update the data structures to ensure the consistency condition.

Logical Time

- Each processor needs a protocol to update the data structures to ensure the consistency condition.
 - Rule 1: Specify how the local logical clock is updated by a process when it executes an event.
 - Rule 2: Specify how a process updates its logical global clock to update its view of the global time and global progress
- all logical clocks systems implement Rule1 and Rule2 to maintain the monotonicity property of causality.

Scalar Time

- Proposed by Lamport in 1978
 - Lamport won the 2013 Turing award for “contributions to the theory and practice of distributed and concurrent systems, notably the invention of concepts such as **causality and logical clocks**,”.
- Time domain is the set of non-negative integers.
- The logical local clock of a process p_i and its local view of the global time are combined into one integer variable C_i .

Scalar Time

- Data structure: The logical local clock of a process p_i and its local view of the global time are squashed into one integer variable C_i .
- Rule1: Before executing an event (send, receive, or internal), process p_i executes the following:

$$C_i := C_i + d \ (d > 0)$$

- Rule 2: Each message piggybacks the clock value of its sender at sending time. When a process p_i receives a message with timestamp C_{msg} , it executes the following actions:
 1. $C_i := \max(C_i, C_{msg})$
 2. Execute Rule1.
 3. Deliver the message.

Scalar Time

- Example. Use $d = 1$ and assign time stamps for the events.

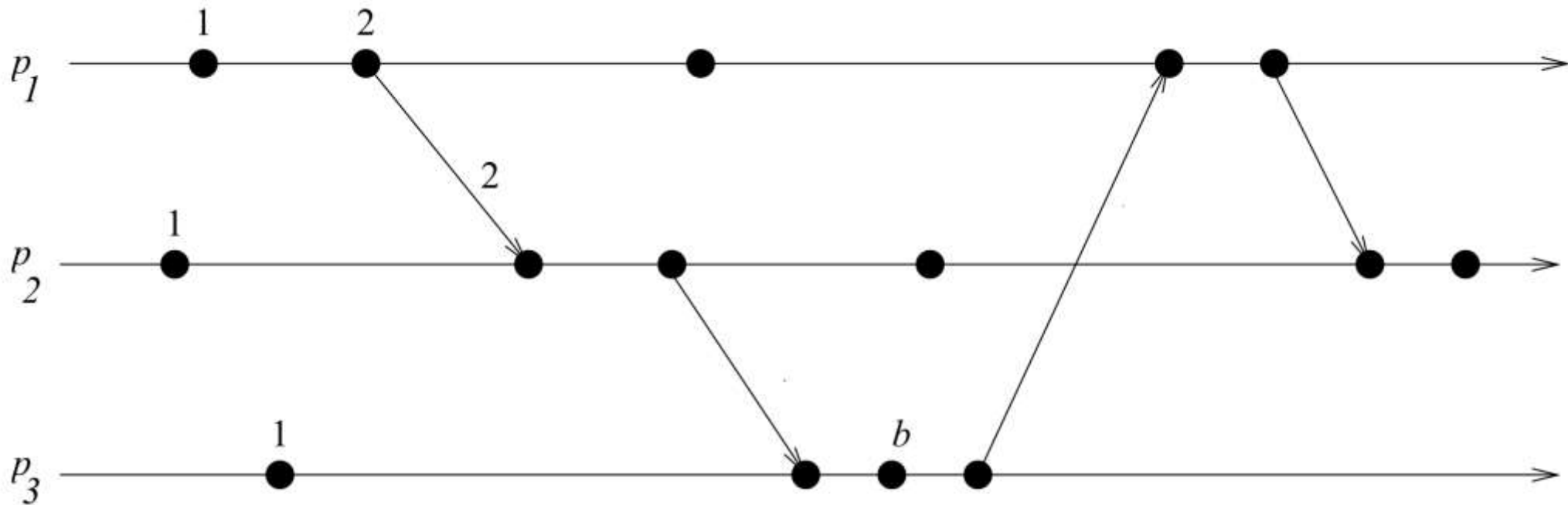


Figure 3.1: The space-time diagram of a distributed execution.

Scalar Time

- Example

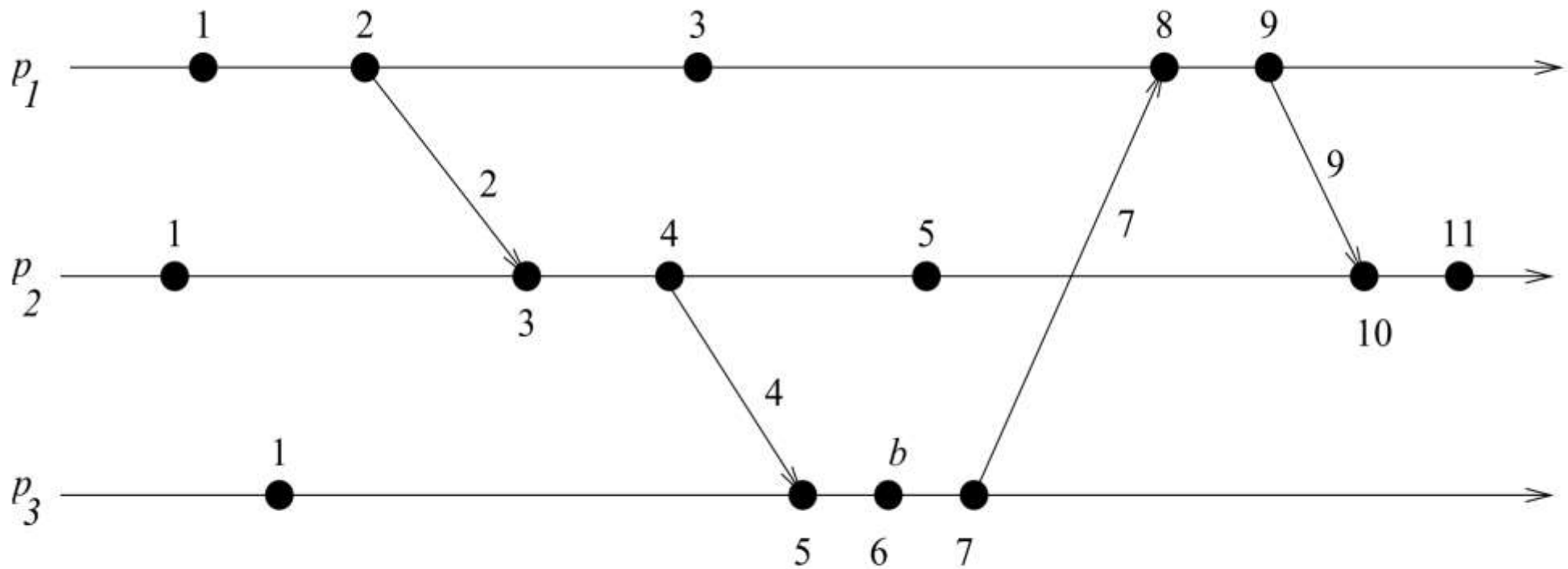


Figure 3.1: The space-time diagram of a distributed execution.

Scalar Time Properties

- **Monotonicity**
 - For two events e_i and e_j , $e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$.
 - **Total Ordering**: Can use the logical time given by the scalar clocks to induce a total order .
 - Note that the timestamps alone do not induce a total order.
 - Two events at different processors can have an identical timestamp.
 - But the tuple (t, i) for each event with $(t_1, i_1) < (t_2, i_2)$ if either $t_1 < t_2$ or $(t_1 == t_2)$ and $i_1 < i_2$ is a total order. In (t, i) , t denotes timestamp and i the identity of the process.
 - This total order is consistent with the relation $\rightarrow$.
- Note: According to the total order above: For events e_1 and e_2 , $e_1 < e_2 \Rightarrow$ Either $e_1 \rightarrow e_2$ or $e_1 \parallel e_2$.**

Scalar Time: Properties

- **Total Ordering:** Tie breaking mechanism. Requests are timestamped and served according to the total order based on these timestamp
- Consider the tuple (t, i) for each event where t denotes timestamp and i the identity of the process.
- $(t_1, i_1) < (t_2, i_2)$ if either $t_1 < t_2$ or $(t_1 == t_2)$ and $i_1 < i_2$ is a total order.

This total order is consistent with the relation $\rightarrow$.

Note: According to the total order above: For events e_1 and e_2 , $e_1 < e_2 \Rightarrow$ Either $e_1 \rightarrow e_2$ or $e_1 \parallel e_2$.

Scalar Time

- Event Counting:
- Strong Consistency:

Scalar Time

- **Event Counting:** Set the increment d to 1 always.
 - If some event e has a timestamp t , then e is dependent on $t - 1$ other events to occur.
 - This can be called as the height of event e .
- **No Strong Consistency:** Note that scalar time does not provide strong consistency. [Strong consistency requires that $e_i \rightarrow e_j \Leftrightarrow C(e_i) < C(e_j)$.]
Example suffices. Refer to the timeline again.

Scalar Time

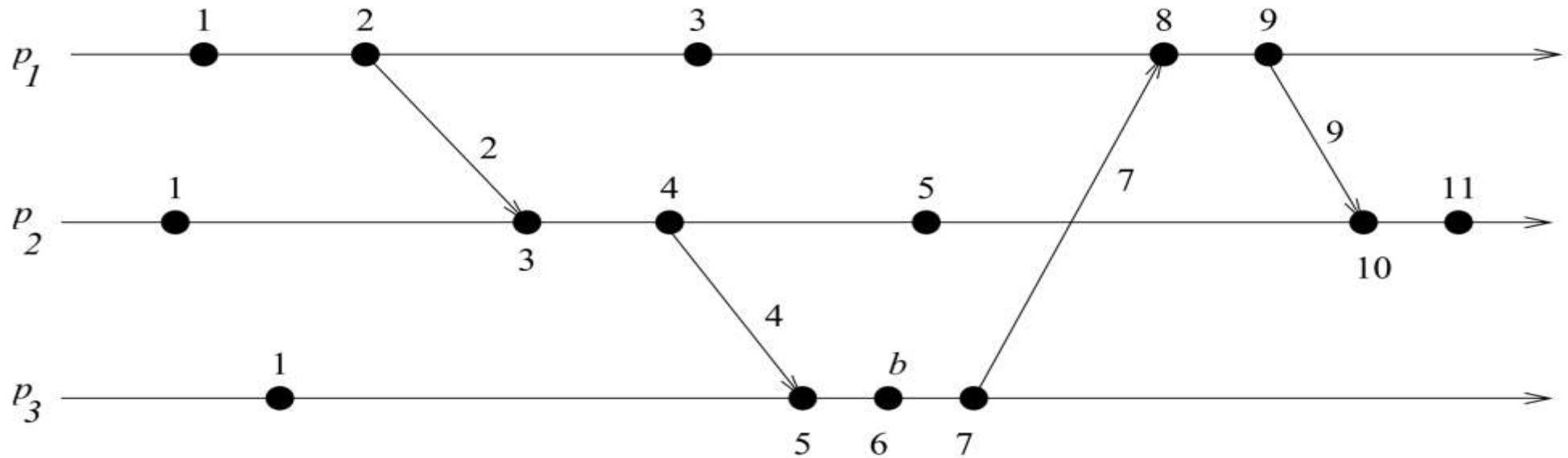


Figure 3.1: The space-time diagram of a distributed execution.

- No Strong Consistency: Consider third event of p_1 and p_2 .

Out of order messages

- First $P1 \rightarrow a \ P3$
- Next $P1 \rightarrow c \ P2$
- Then, $P2 \rightarrow b \ P3$

At P3, it is possible that b arrives before a (though a was sent before).

When b arrives:

- P3 does not know whether a msg with lower timestamp might arrive
- Even if a arrives with lower timestamp, P3 can not know whether a actually happened before b as b is coming from P2 (following its own count) and a has come from P1.

Scalar Time: Limitation

- One of the limitations of scalar time is the lack of strong consistency.
- Strong consistency not achieved as scalar time uses a single time that represents the logical local clock and the logical global clock.
 - This means that the causality of events across processors is lost. That is, by looking at the times we can not make out which two events are casually related and which two are not.
 - Vector time solves this problem but with big data structures.

Vector Time

- Time is represented by n -dimensional non-negative integer vectors.
- Each process p_i maintains a vector $vt_i [1..n]$, where $vt_i[i]$ is the local logical clock of p_i and describes the logical time progress at process p_i .
- $Vt_i[j]$ represents process p_i 's latest knowledge of process p_j local time.
- If $vt_i [j]=x$, then process p_i knows that local time at process p_j has progressed till x .
- The entire vector vt_i constitutes p_i 's view of the global logical time and is used to timestamp events.
- The timestamp of an event is the value of the vector clock of its process when the event is executed.

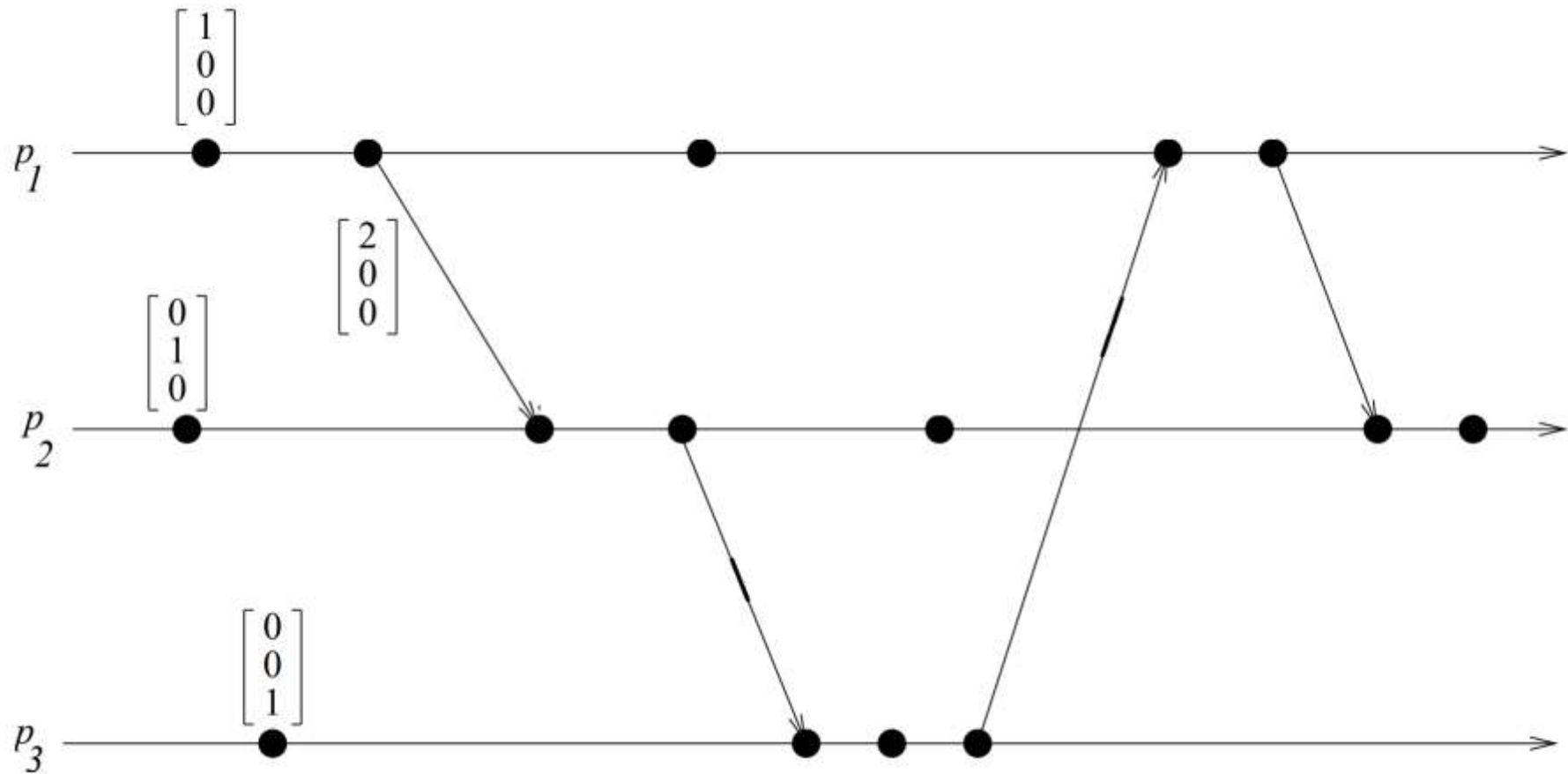
Vector Time – Update Rules

Rules for Process p_i to update its clock:

- Rule 1: Before executing an event, process p_i updates its local logical time as follows:
$$vt_i[i] := vt_i[i] + d \text{ where } (d > 0)$$
- Rule2: Each message m is piggybacked with the vector clock vt of the sender process at sending time. On the receipt of such a message (m, vt) , process p_i executes the following sequence of actions:
 1. Update its global logical time as follows:
 2. $1 \leq k \leq n : vt_i[k] := \max(vt_i[k], vt[k])$
 3. Execute Rule1.
 4. Deliver the message m .

Vector Time – Example

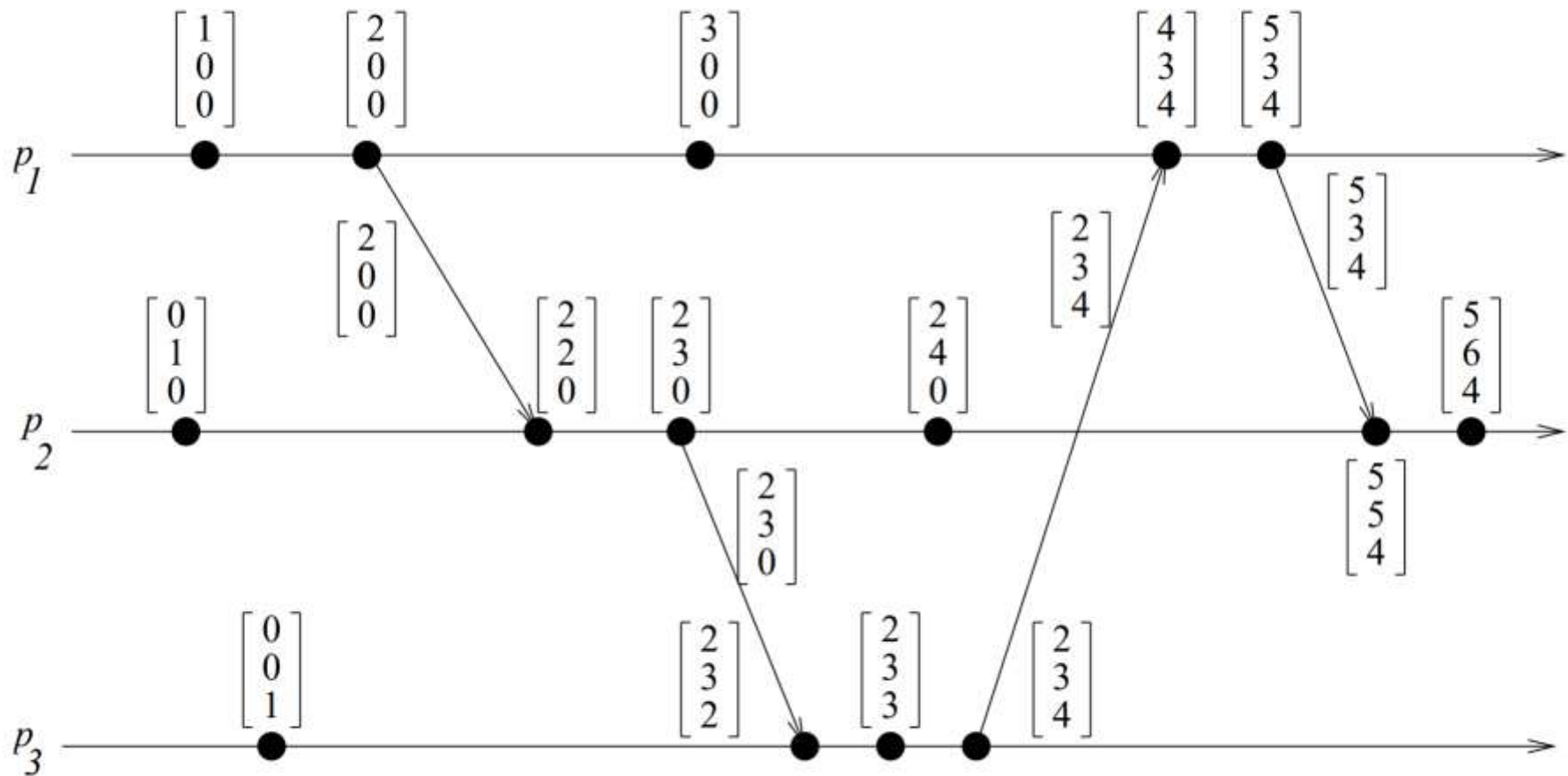
P2's second event:
 $\max([2,0,0], [0,1,0])=[2,1,0]$
Then Rule1: $[2,2,0]$



-
- For process i , for $d=1$, are the events internal to the process ordered as a sequence $1, 2, 3, 4, \dots$. Or could there be jumps ?

Vector Time – Example

- Initially, a vector clock is $[0, 0, 0, \dots, 0]$. $d = 1$.



Vector Time – Example

- Using vector clocks, two timestamps vh and vk are compared as follows.
- $vh == vk$ iff for all indices i , $vh[i] == vk[i]$
- $vh \leq vk$ iff for all indices i , $vh[i] \leq vk[i]$
- $vh < vk$ iff $vh \leq vk$ and there exists an index i where $vh[i] < vk[i]$.
- $vh \parallel vk$ iff $\text{not}(vh < vk)$ and $\text{not}(vk < vh)$

Properties of Vector Time

- **Event Counting:** Use $d = 1$ always.
- Then the i th component of vector clock at process p_i , $vt_i[i]$, denotes the number of events that have occurred at p_i until that instant.
- So, if an event e has timestamp vh , $vh[j]$ denotes the number of events executed by process p_j that causally precede e .

How many events causally precede e in the distributed computation ?

Properties of Vector Time

- **Event Counting:** Use $d = 1$ always.
- Then the i th component of vector clock at process p_i , $vt_i[i]$, denotes the number of events that have occurred at p_i until that instant.
- So, if an event e has timestamp vh , $vh[j]$ denotes the number of events executed by process p_j that causally precede e . Clearly, $\left(\sum_j vh[j]\right) - 1$ represents the total number of events that causally precede e in the distributed computation.

Properties of Vector Time

Are Vector clocks strongly consistent ?

Properties of Vector Time

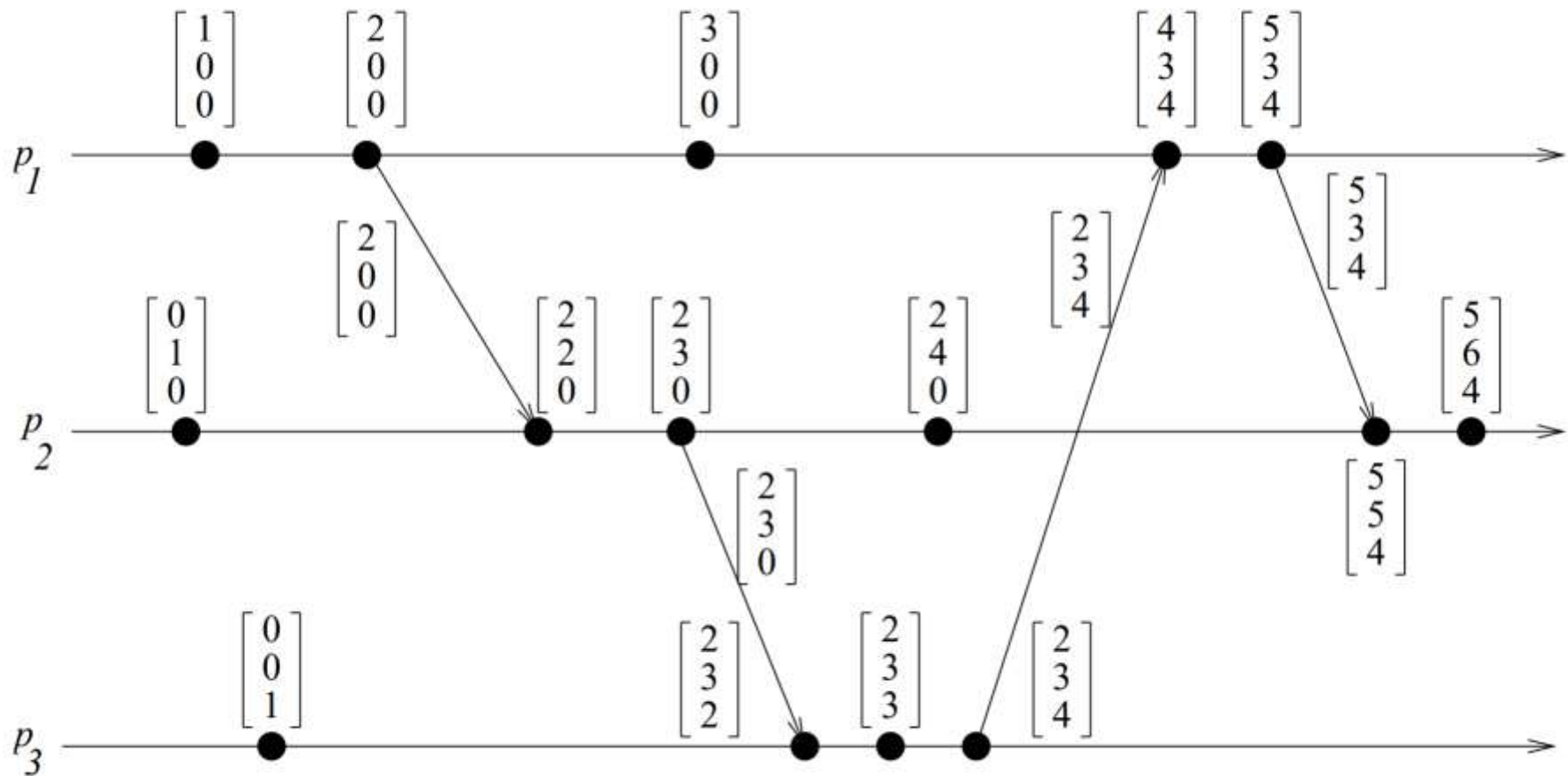
- **Strong Consistency:** Vector clocks are strongly consistent.
- What are the limitations of the vector clock?

Limitations of Vector Time

- Large message sizes owing to the vector being piggybacked on each message.
- The message overhead grows linearly with the number of processors in the system and when there are thousands of processors in the system, the message size becomes huge even if there are only a few events occurring in few processors.
- Few techniques exist to reduce the overhead.
- What can we do to reduce the load?

Vector Time – Example

- Initially, a vector clock is $[0, 0, 0, \dots, 0]$. $d = 1$.



Optimizations

- Consider a message from P_i to P_j .
- The message includes the vector time at P_i .
- Of the n components of the vector time, only few would have changed since the last time P_i sent a message to P_j .
- Need to include only those tuples.
 - $(i_1, t_1), (i_2, t_2), \dots$
- Update rule changes slightly so that:
 - P_j updates the component k of its vector time to $\max\{vt_j[k], tk\}$ where tk is the time reported by P_i with the tuple (k, tk)

Are we missing something?

Optimizations

- Message sizes decrease in general, but local storage may increase.
 - Need to know the vector time at P_i when the last message was sent to P_j , for each j .

That is, each process stores the last msg sent to every other process.

- $O(n^2)$ storage per process.
- A clever optimization can bring this storage to $O(n)$ – Singhal kshemkalyanis technique

Singhal- Kshemkalyani Differential Technique

- Process p_i maintains the following two additional vectors:

$\Delta LS_i[1..n]$ ('Last Sent') : $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to process p_j .

$\Delta LU_i[1..n]$ ('Last Update') : $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.

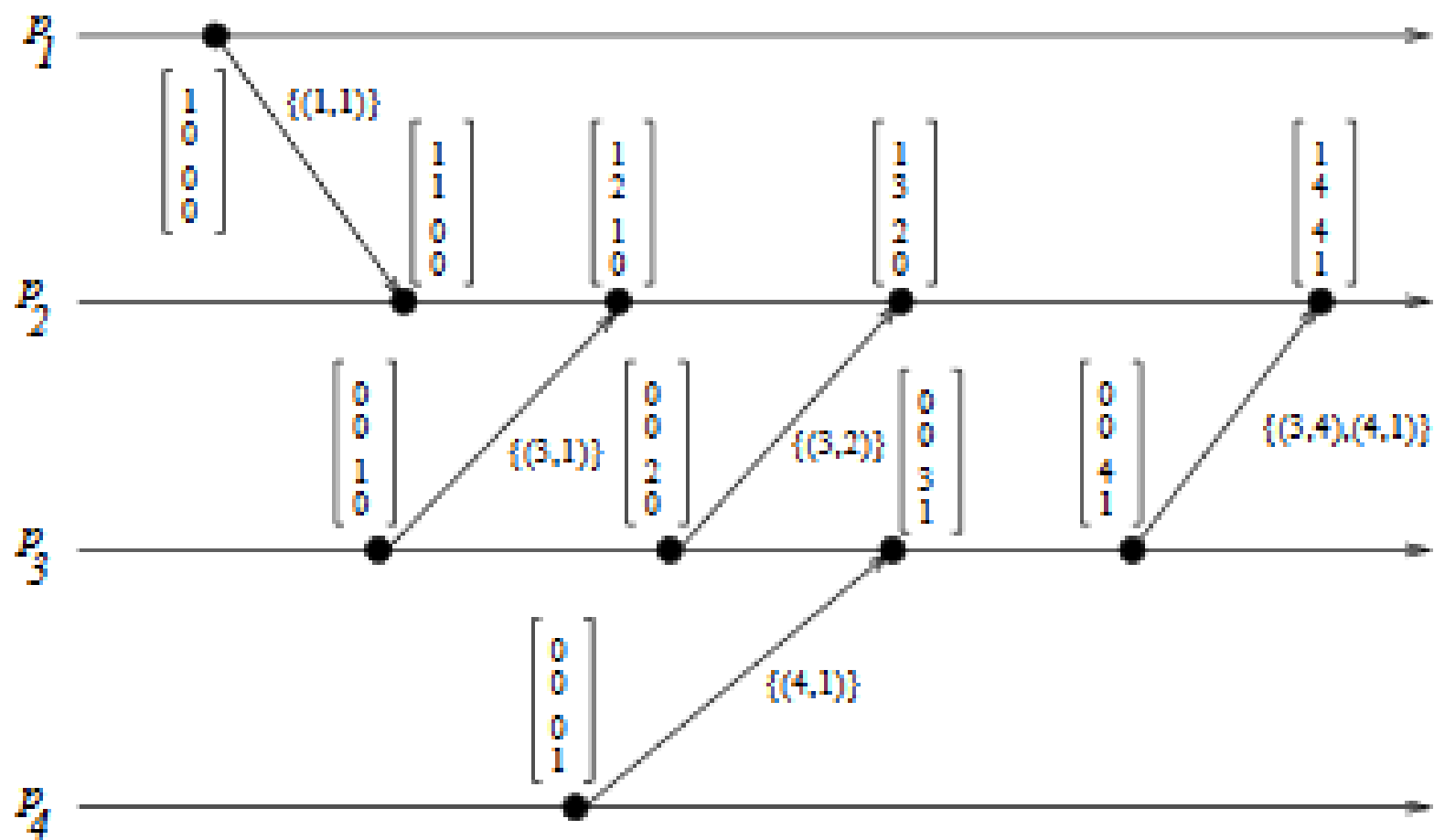
$LU_i[i] = vt_i[i] ?$

When do you update LS and LU ?

Can we put LS and LU together to know what is the information that needs to be piggybacked on the message ?

Singhal- Kshemkalyani Differential Technique

- Process p_i maintains the following two additional vectors:
 - $\Delta LS_i[1..n]$ ('Last Sent') : $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to process p_j .
 - $\Delta LU_i[1..n]$ ('Last Update') : $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.
- Clearly, $LU_i[i] = vt_i[i]$ at all times
- $LU_i[j]$ needs to be updated only when the receipt of a message causes p_i to update entry $vt_i[j]$.
 - $LS_i[j]$ needs to be updated only when p_i sends a message to p_j



Singhal- Kshemkalyani Differential Technique

- Since the last communication from p_i to p_j , only those elements of vector clock $vt_i[k]$ have changed for which $LS_i[j] < LU_i[k]$ holds. Hence, only these elements need to be sent in a message from p_i to p_j .
- When p_i sends a message to p_j , it sends only a set of tuples $\{ (x, vt_i[x]) \mid LS_i[j] < LU_i[x] \}$ as the vector timestamp to p_j , instead of sending a vector of n entries in a message.

Home Work !!

Think about whether vector clocks can detect out of order msgs.

Will Singhal-Kshemkalyanis technique work if the messages are not in FIFO order ?



Understanding Vector Clocks further

Teasers-1

- Suppose I want to know whether an updation event at site D at time t which was propagated(broadcasted) to all sites has reached everyone.
- **Nodes need this information to determine when it is safe to discard/stabilize updates.**
- Can I decipher looking at B's vector sent to me if B has received the msg from D?
 - Conditions: Broadcast by D, I have received the broadcast, no msg drop, several other msgs are being passed in the system simultaneously.

Teasers-1

- Suppose B received a msg from C before it sent a msg to me then

B updates as

$$V_B[D] = \max(V_B[D], V_C[D]).$$

Hence we do not know if $V_B[D]$ is update from C or an update B already had from D.

Teasers-2

- Suppose I want to know whether an updation event at site D at time t which was propagated to all sites has reached everyone assuming causal order and no lost messages:

Consider the below:

B sends msg to me. Can I know if B received the msg from D?

Conditions: Causal Order, Broadcast by D, I have received the broadcast,

- no msg drop, several other msgs are being passed in the system simultaneously.